Attention Is All You Need

NIPS 2017 (now NeurIPS) · 2017 · Ashish Vaswani et al.

Abstract

Machine translation maps a source-language sequence to a target-language sequence. To reduce recurrent models' sequential training dependencies and long paths between distant positions, the paper introduces the Transformer: multi-head attention exchanges contextual information, while feed-forward networks and positional encodings support encoding and decoding. On the WMT 2014 newstest2014 test set, Table 2 reports English-to-German scores of 27.3 BLEU for base and 28.4 for big, compared with 25.16 for ConvS2S. For English-to-French, big achieves 41.8 BLEU, versus 40.46 for ConvS2S. Higher BLEU is better.

Notation

Symbol	Meaning	Type / shape	Role	Source / scope
Q	Query matrix	$n \times d_k$	Specifies what a position is looking for	Section 3.2.1; single-head self-attention
K	Key matrix	$n \times d_k$	Matches queries to produce relevance scores	Section 3.2.1; single-head self-attention
V	Value matrix	$n \times d_v$	Supplies information for the weighted sum	Section 3.2.1; single-head self-attention
d_{model}	Model representation width	512 in base	Common input/output width across layers	Section 3.1
d_k	Query/key width per head	64 in base	Sets matching dimension and dot-product scaling	Section 3.2.2
d_v	Value width per head	64 in base	Sets each head's output width	Section 3.2.2
h	Number of attention heads	8 in base	Runs several matching operations in parallel	Section 3.2.2

1 Prerequisites

This report covers the original encoder-decoder Transformer for machine translation. First review seven prerequisite concepts, then follow the architecture and tensor shapes. In the notation table, n is the source sequence length. Indices, temporary dimensions, and simple operations are explained where they appear.

1.1 Sequence transduction and the encoder-decoder framework

Concept: **Sequence transduction** maps one sequence to another, such as English to German. An **encoder-decoder** separates representing the source sentence from generating the target sentence: the encoder builds contextual representations, and the decoder produces target tokens step by step.

Mechanism: the decoder predicts the next token using both the source representation and the known target prefix. The training objective shapes how the two components cooperate, so

their representations are jointly constrained by the translation task.

Example: when translating the Chinese equivalent of “I like learning” into English, the prediction of “like” can use the source sentence and the already generated “I.” This paper retains that overall framework and changes how positions within a sequence exchange information.

1.2 Sequential computation in RNNs and LSTMs

Concept: an **RNN** hidden state depends on the preceding state. **LSTMs** and **GRUs** use gates to retain information more effectively, but keep this temporal dependency.

Mechanism: within one training example, a later position must wait for the preceding position. This limits parallelism along the sequence, although examples and some matrix operations can still run in parallel. Information between distant positions also passes through multiple recurrent states.

Example: in “The shop that I visited with friends last month and that was later renovated has closed,” early descriptive information must travel across many positions. Table 1 gives a recurrent layer a maximum path length of $O(n)$ and fully connected self-attention a path length of $O(1)$. These quantities describe information-transfer paths; task performance is evaluated by the experiments.

1.3 The basic idea of attention

Concept: **attention** scores how a query matches candidate keys, normalizes the scores, and aggregates values with the resulting weights. Q specifies what to look for, K provides matching features, and V supplies content.

Mechanism: the weights depend on the input, and the operation is differentiable. Translation loss can train the projection parameters to make task-relevant information easier to retrieve. Attention weights are not manually assigned importance labels.

Example: if three candidate values receive weights of 0.7, 0.2, and 0.1, the output is their weighted sum, with the first contributing most. The paper uses this operation both for interaction within a sequence and for alignment between a source sentence and its translation.

1.4 Self-attention

Concept: in **self-attention**, queries, keys, and values come from the same sequence, usually through different learned projections. They are therefore not identical arrays.

Mechanism: a layer uses its fixed input representations to compute all position-to-position matches and weighted sums simultaneously. A position can read another distant position directly, without a recurrent word-by-word computation inside that layer.

Example: the relevant words in “There is a river beside the bank” and “A car is parked on the riverbank” can acquire different representations through their contextual weights. The encoder can read the entire source sentence. Decoder self-attention uses a causal mask and can read only the known target prefix.

1.5 Embeddings and positional information

Concept: an **embedding** maps a token ID to a vector; a **positional encoding** adds ordering information. The base model uses embedding width $d_{model} = 512$.

Mechanism: without positional encodings or position-dependent constraints, self-attention is equivariant to a permutation of its inputs and cannot recover the original absolute word order from this operation alone. Positional encodings give the same token different input representations at different positions.

Example: “I help you” and “You help me” contain the same words but require different translations. This paper uses fixed sine and cosine encodings. Learned positional embeddings obtain similar results in Table 3, showing that the tested setup permits more than one positional representation.

1.6 Residual connections and layer normalization

Concept: a **residual connection** adds the input directly to a sublayer's output. **Layer normalization (LayerNorm)** normalizes over each position's feature dimension and has learned scale and shift parameters.

Mechanism: the residual path provides a relatively direct route for information and gradients, while normalization controls intermediate numerical scales. Together they support optimization of deeper networks.

Example: a sublayer can learn an increment to the existing representation rather than reconstructing it from scratch. The paper uses **Post-LN**: residual addition precedes normalization. Section 3.1 writes this as:

$$LayerNorm(x + Sublayer(x)) \quad (1)$$

1.7 Machine translation and BLEU

Concept: the paper evaluates **WMT 2014** English-to-German and English-to-French translation, using approximately 4.5 million and 36 million training sentence pairs. **BLEU** measures n-gram overlap with reference translations and applies a brevity penalty. Higher is better.

Mechanism: an automatic metric enables consistent comparison across many translations, but tokenization, test data, and decoding settings affect the result. Semantically valid alternative wording may receive limited credit.

Example: two translations with similar meanings but different wording can receive different scores, so interpret differences under the same evaluation protocol. The main results use newstest2014, while the ablations use newstest2013; the two tables are not results on the same test set.

2 Motivation and publication

2.1 Publication details

Ashish Vaswani and 7 coauthors published the paper at **NIPS 2017**, now called NeurIPS. This report uses arXiv:1706.03762v7, whose first page records an update on 2 August 2023.

That update date is not the original publication year. All page, table, and figure numbers below refer to this 15-page PDF.

2.2 Problem and motivation

Sections 1–2 aim to reduce sequential dependencies in sequence processing. Recurrent models process time steps within an example; convolutional models can process several positions together, but distant relationships generally pass through multiple layers of local combination. The authors seek direct access to distant information while retaining computation suited to matrix operations. Autoregressive inference still generates target tokens step by step.

2.3 Sources of inspiration

Attention-based alignment, convolutional sequence models, earlier self-attention, and memory networks form the background. The Transformer organizes these ideas into a sequence-transduction architecture built from multi-head self-attention, cross-attention, and position-wise feed-forward networks. “All You Need” refers to replacing the recurrent and convolutional structures used for sequence processing. The model still includes embeddings, linear layers, nonlinearities, normalization, and positional encodings.

3 Method

3.1 Start with the baselines and the actual change

Distinguish two kinds of baseline. ConvS2S and GNMT in Table 2 are external experimental comparators; Transformer base in Table 3 is the ablation reference. The former compare complete systems, while the latter helps analyze particular configurations. We focus on **ConvS2S**, reference [9] in the main paper, because it also pursues parallel sequence modeling and makes the core difference easy to explain.

3.1.1 ConvS2S and Transformer core mechanisms

ConvS2S stacks one-dimensional convolutions in the encoder and decoder, using gated linear units and residual connections. Its local context expands with depth, and every decoder layer also attends to the source sentence. Transformer replaces convolutional mixing within a sequence with self-attention and runs multiple matching operations in parallel. Source: Gehring et al., Sections 3.2–3.3; see the reference list.

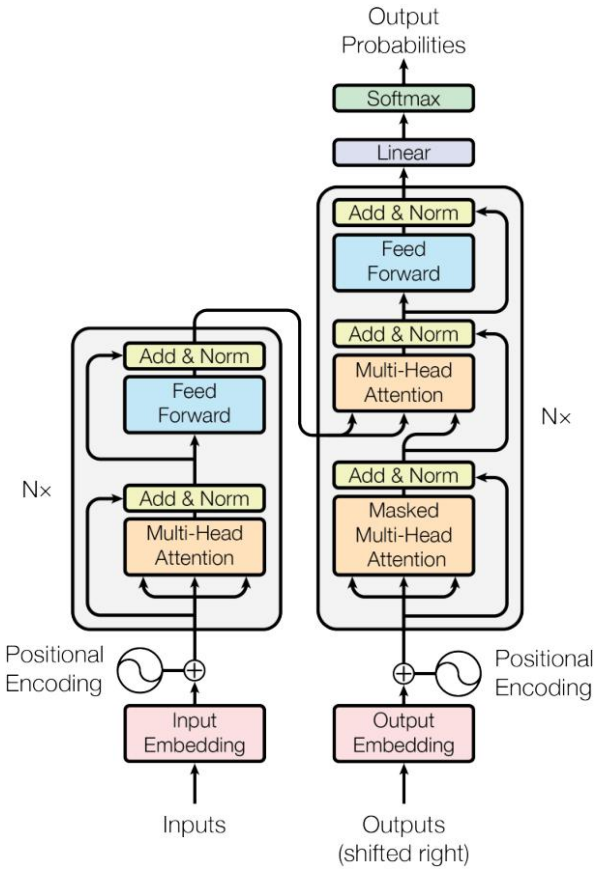
Aspect	ConvS2S	Transformer
Within-sequence interaction	Stacked local convolutions	Self-attention directly connects visible positions
Reading the source	Source attention in each decoder layer	Multi-head cross-attention in each decoder layer
Position information	Learned positional embeddings	Fixed sinusoidal encodings by default
Generation	Autoregressive, one token at a time	Also autoregressive, one token at a time

3.1.2 How to interpret the comparison

Table 2 groups reported WMT 2014/newstest2014 results to compare complete systems. Data processing, model size, and training budgets are not controlled to be identical, so the score differences reflect system-level differences. To assess the effect of a particular internal change, focus on the variants evaluated under the common development-set setup in Table 3.

3.2 Overall architecture: encoder and decoder stacks

The base encoder and decoder each contain 6 layers. Here $N = 6$ denotes repetition of the architecture; different layers have independent parameters. Each encoder layer contains multi-head self-attention and a position-wise feed-forward network. A decoder layer adds cross-attention to read the encoder output. Residual connections and LayerNorm surround the sublayers. Their external width is consistently $d_{model} = 512$, allowing residual addition. Source: Section 3.1.



Caption: Original Figure 1 (p. 3): the Transformer encoder and decoder.

Read Figure 1 from bottom to top. On the left, Inputs pass through embeddings and are added to positional encodings. Inside the gray box, multi-head self-attention is followed by Add & Norm, Feed Forward, and another Add & Norm. The three incoming attention arrows indicate that queries, keys, and values all originate from the layer input but have separate projections. Arrows bypassing the sublayers are residual paths.

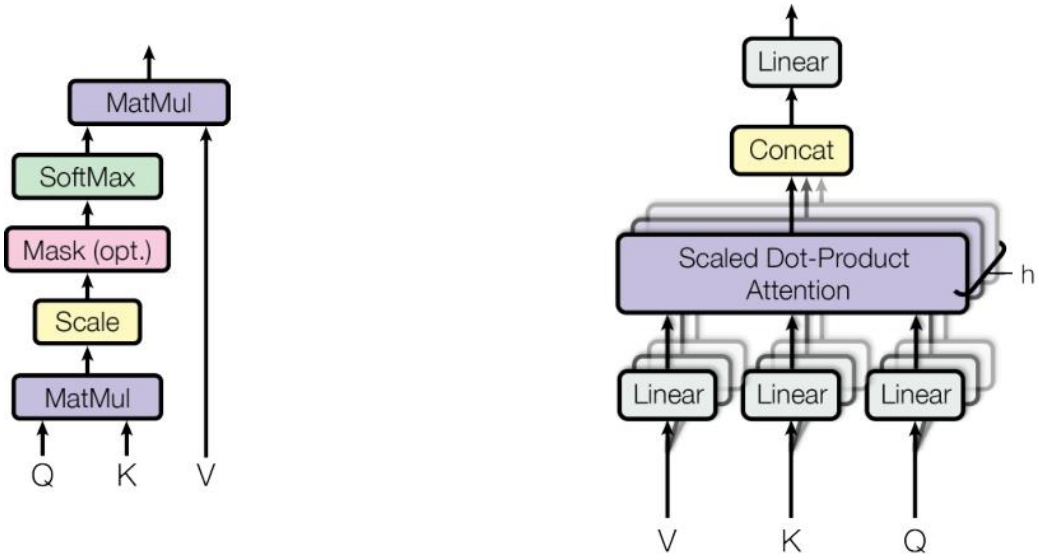
On the right, Outputs (shifted right) denotes the target sequence shifted by one position during training. Masked Multi-Head Attention hides future positions. The next Multi-Head Attention block receives queries from the decoder's updated self-attention representation, while keys and values come from the final encoder output. A feed-forward sublayer follows. At the top, Linear and Softmax convert representations into probabilities over the target vocabulary. N× indicates repeated stacking.

3.3 Scaled dot-product attention

Given queries, keys, and values, scaled dot-product attention computes:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2)$$

QK^T scores each query against the keys; T denotes matrix transposition. A row-wise softmax produces weights, which then aggregate V. A footnote motivates scaling under a simplified assumption: if query and key components are independent, with mean 0 and variance 1, the dot-product variance grows with d_k. Dividing by $\sqrt{d_k}$ controls the scale and helps avoid excessive softmax saturation. Scaling addresses the magnitude of the scores; the optimizer, residual connections, and normalization also influence training. Source: Section 3.2.1.



Caption: Original Figure 2 (p. 4): scaled dot-product attention and multi-head attention.

The left diagram shows matrix multiplication, scaling, an optional mask, softmax, and multiplication by the value matrix. Masking must occur before softmax: a hidden position receives negative infinity, giving it zero normalized weight. The right diagram runs this process multiple times in parallel. Each head first applies its own projections; the attention outputs are concatenated and then passed through an output projection. Independent projection parameters allow the heads to learn different matching patterns.

3.4 Multi-head attention

Multi-head attention learns several low-dimensional projections. Each head independently matches and aggregates:

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V) \quad (3)$$

$$MultiHead(Q, K, V) = Concat(head_1, \ldots, head_h)W^O \quad (4)$$

Here i identifies the head, W_i^Q, W_i^K, W_i^V are its projections, and W^O projects the concatenated outputs. Base uses $h = 8$ and $d_k = d_v = 64$: each head learns a 64-dimensional view of the 512-dimensional input, and the 8 outputs concatenate to width 512. The paper explains that this reduced per-head width keeps total computation similar to a full-width single head. Actual cost includes projections, tensor organization, and attention-matrix computation.

Heads may learn different relationships; their roles are not assigned in advance. Source: Section 3.2.2.

3.5 Three uses of attention in the model

The three connections differ in information source and visibility. Encoder self-attention obtains queries, keys, and values from the same encoder-layer input and can access every source position. Decoder masked self-attention obtains all three from its decoder-layer input, but can read only the current and preceding input positions. Because the target input is shifted, a prediction cannot see its own answer. Cross-attention obtains queries from the representation after that layer's decoder self-attention, and keys and values from the encoder stack's final output. Each decoder layer can therefore reread the source sentence. Source: Section 3.2.3.

3.6 Position-wise feed-forward network

Attention exchanges information between positions. The **feed-forward network (FFN)** performs nonlinear processing independently at each position:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (5)$$

Here x is the current position's row vector, W_1, W_2 are weights, and b_1, b_2 are biases. Base uses inner width $d_{ff} = 2048$, so the shape changes from 512 to 2048 and back to 512. The 2048 features are intermediate features within this sublayer. Parameters are shared across positions in a layer, but differ across layers. Expanding and then compressing the width enables nonlinear feature combinations in a larger intermediate space. Source: Section 3.3.

3.7 Embeddings, positional encodings, and output projection

Token embeddings are multiplied by $\sqrt{d_{model}}$ and added to positional encodings of the same width. The sinusoidal encodings are:

$$PE_{(pos,2i)} = \sin\left(pos/10000^{2i/d_{model}}\right) \quad (6)$$

$$PE_{(pos,2i+1)} = \cos\left(pos/10000^{2i/d_{model}}\right) \quad (7)$$

pos is a position; here i indexes a pair of dimensions, rather than a head as in the multi-head equations. The different frequencies give position offsets a structured representation. One motivation for this choice was the possibility of extending to longer sequences. Sections 5.5.3–5.5.4 examine long-context applicability using later research.

The decoder's final linear layer projects to the vocabulary, followed by softmax probabilities. Section 3.4 describes sharing weights between the two embedding layers and the output projection. English-to-German uses a shared vocabulary of approximately 37,000 BPE subwords; English-to-French uses approximately 32,000 word pieces.

3.8 Training: optimizer and regularization

Training uses source-target sentence pairs. The correct target sequence, shifted right, becomes the decoder input; the model predicts the correct token at each position using cross-entropy with label smoothing. Embeddings, attention projections, FFNs, the output layer, and learned LayerNorm parameters all receive updates in this supervised training of the complete model. Fixed positional encodings and causal masks have no learned parameters.

Adam uses $\beta_1 = 0.9$, $\beta_2 = 0.98$, and $\varepsilon = 10^{-9}$. The learning-rate schedule is:

$$lrate = d_{model}^{-0.5} \min(s^{-0.5}, s \cdot w^{-1.5}) \quad (8)$$

s is the training step and $w = 4000$ the number of warmup steps. The rate first increases linearly and then decays as the inverse square root of the step count. Base uses residual dropout of 0.1 and label smoothing of 0.1. On 8 P100 GPUs, base trains for 100,000 steps in about 12 hours; big trains for 300,000 steps in about 3.5 days.

Inference uses beam search with beam size 4 and length penalty 0.6. Base averages the last 5 checkpoints and big the last 20. This averages parameters from different training times of one model; Ensemble entries in Table 2 instead combine several models. Sources: Sections 5 and 6.1.

4 How the input changes

Follow one batch through the **base configuration**. Let batch size be B, source length n, and target length m. The illustrative example uses $B = 2$, $n = 6$, and $m = 5$ to demonstrate tensor shapes; these are not dataset statistics. Training has the complete correct target sequence, while inference has only a growing target prefix.

4.1 Step 1: sentences become position-aware vector sequences

Split sentences into tokens, convert them to integer IDs, and pad the batch to obtain an ID array of shape $B \times n$. An embedding lookup produces $B \times n \times 512$. Multiply by $\sqrt{512}$ and add the sinusoidal vector for each position; the shape does not change. Training teaches the embedding table which vectors help predict translations, while fixed sinusoidal encodings remain unchanged. Attention should mask padded positions so padding tokens are not treated as real words.

4.2 Step 2: six encoder layers update contextual representations

4.2.1 Shapes inside the self-attention sublayer

Each layer projects its $B \times n \times 512$ input several ways. In batch-head-position-feature order, Q, K, V each have shape $B \times 8 \times n \times 64$. Within a head, QK^T produces an $n \times n$ score matrix, giving $B \times 8 \times n \times n$ across the batch. After scaling and normalizing across key positions, the weights multiply V, returning $B \times 8 \times n \times 64$. Concatenate the 8 heads and apply W^O to obtain $B \times n \times 512$.

In the illustrative batch, each head has a 6-by-6 weight map, and there are 2-by-8 such maps. A row identifies the querying position; a column identifies the position it reads. The learned objects are the projection parameters. Attention weights are intermediate results computed from the current input and parameters, not a permanently fixed table independent of the input.

4.2.2 Position-wise feature processing in the FFN

Residual addition and LayerNorm retain shape $B \times n \times 512$. The FFN expands this to $B \times n \times 2048$, applies ReLU, and projects back to $B \times n \times 512$, followed by another residual addition and normalization. This structure repeats across 6 layers with separate parameters. It

progressively updates contextual representations; attention visualizations and ablations help analyze the relationships formed at different layers.

4.3 Step 3: the decoder reads its prefix, consults the source, and processes features

During training, the correct target sequence is shifted right to form $B \times m$ IDs and then $B \times m \times 512$ embeddings. Decoder self-attention has weight shape $B \times 8 \times m \times m$. Future input positions are masked; weights for visible positions are still computed by the model. Because the input is shifted, predicting a target token exposes only earlier target tokens.

Cross-attention has query length m and key/value length n, giving a weight map of shape $B \times 8 \times m \times n$. In the example, each head has a 5-by-6 map: 5 target positions query 6 source positions. Queries come from the decoder representation just processed in the current layer; keys and values project the final encoder representation. Aggregation returns $B \times m \times 512$, followed by the FFN, residual connections, and normalization.

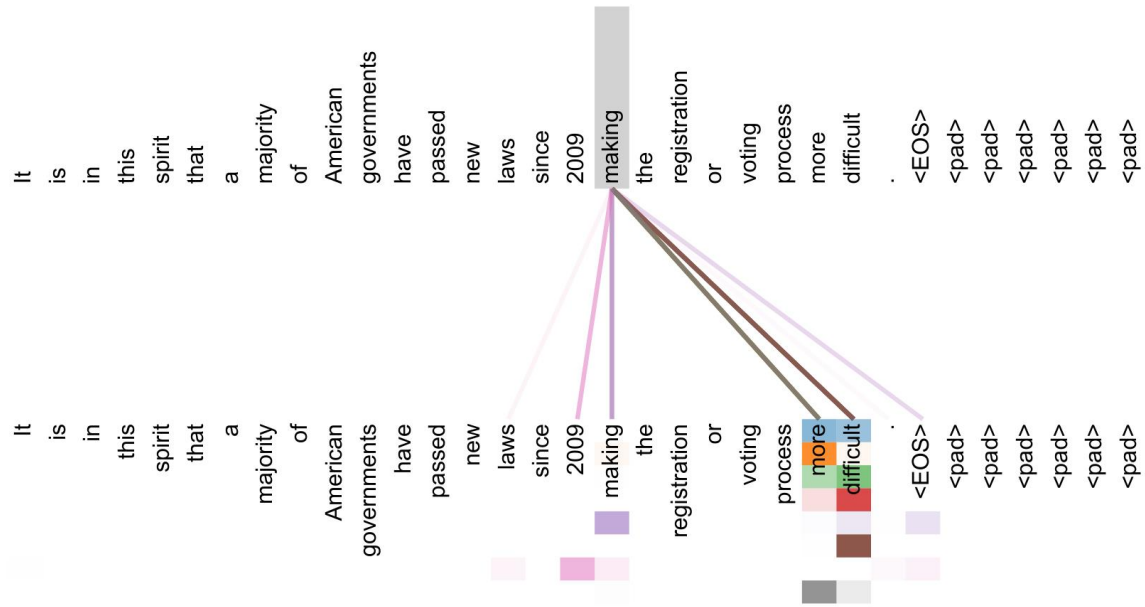
4.4 Step 4: from 512 features to the next token

Let vocabulary size be U. The output linear layer maps $B \times m \times 512$ to logits of shape $B \times m \times U$. During training, losses for all target positions can be computed together, and backpropagation updates the entire network. Differences between predicted probabilities and the correct tokens affect upstream parameters through the output layer, decoder, cross-attention, and encoder.

At inference time, the complete correct target sequence is unavailable. Starting with a start token, the decoder predicts from its generated prefix. Beam search retains several candidate sequences until an end token or length limit is reached. Encoder outputs can be reused, and incremental decoding can cache past projections to avoid recomputation. Target tokens are still generated sequentially.

4.5 What attention has learned

The appendix visualizes attention from the trained model. The following figure shows weights only for the query word “making” in encoder layer 5. Colors distinguish heads; the connections and corresponding colored marks indicate which source positions those heads read. Focus on the connected positions in this example and how those connections support contextual aggregation.



Caption: Original Figure 3 (p. 13): encoder layer 5 attention for the query word making.

The upper and lower rows show the tokens of the same sentence. Clear connections link “making” to the later words “more” and “difficult,” capturing the long-distance relation in “making … more difficult.” Other heads assign weight to “making” itself and to other positions. This illustrates direct access to distant information within one self-attention layer. To investigate head specialization further, one could measure patterns across sentences and examine prediction changes when individual heads are removed.

4.6 Connect the complete data flow

Source tokens → embeddings and positions → 6 encoder layers → contextual source representations. Target prefix → masked self-attention → cross-attention reading the source → FFN → vocabulary probabilities. During training, errors across all prediction positions jointly update the parameters; inference continues one target step at a time. Understanding Transformer requires separating how positions exchange information, how each position processes features, and how target tokens are generated over time.

5 Contributions and transferable lessons

5.1 Core contributions and reusable design choices

First, stacked self-attention handles within-sequence interaction, reducing recurrent position-by-position dependencies while retaining an encoder-decoder framework and autoregressive outputs. Second, several low-dimensional attention heads operate in parallel, allowing the same input to support different matching relationships. Third, scaled dot products, residual connections, normalization, FFNs, and positional encodings form a trainable complete system. Together these components constitute the paper's architecture.

5.2 Main experimental results and the original table

Table 2 lists models on the left and BLEU for English-to-German (EN-DE) and English-to-French (EN-FR) in separate columns; higher is better. Training cost is estimated in FLOPs. First distinguish single models, ensembles, and the Transformer base/big configurations, then

compare within the same translation direction.

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

Caption: Original Table 2 (p. 8): translation test-set BLEU and estimated training cost.

English-to-German: ConvS2S scores 25.16; MoE, the strongest listed earlier single model, scores 26.03; and the ConvS2S ensemble scores 26.36. Transformer base achieves 27.3 and big 28.4. English-to-French: ConvS2S scores 40.46, its ensemble 41.29, and Transformer big 41.8. Base scores 38.1, below those English-to-French comparators. The advantage therefore depends on model configuration and translation direction; big has stronger results in both directions.

Estimated training cost is 3.3×10^{18} FLOPs for base and 2.3×10^{19} for big, versus 9.6×10^{18} for English-to-German ConvS2S. Base is below that comparator; big is above it. Costs are inferred from training time, GPU count, and estimated throughput. They describe system-level training cost; latency on identical hardware requires a separate measurement.

Source-version note: this v7 PDF reports 41.8 for English-to-French big in both its abstract and Table 2, but one sentence in Section 6.1 says 41.0. This explanation follows the abstract and original table with 41.8 and records the internal discrepancy so readers can check it.

5.3 Ablations and the base reference

Table 3 uses the English-to-German development set newstest2013. Base achieves 25.8 BLEU with approximately 65 million parameters. Each group changes different settings; blank cells inherit base values. These experiments do not use the checkpoint averaging employed for the main results. Compare each variant with the base row of this table to retain the same data and evaluation setting.

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

Caption: Original Table 3 (p. 9): architecture and training variants on the English-to-German development set.

Group A: 1 head scores 24.9; the 8-head base scores 25.8; 16 heads score 25.8; and 32 heads score 25.4. Per-head dimensions also change with the head count. Group B: reducing key dimension to 16 gives 25.1, while 32 gives 25.4. Group C: reducing depth from 6 to 2 gives 23.7. Model width 256 gives 24.5 and width 1024 gives 26.0. Increasing FFN width to 4096 gives 26.2 while also increasing the parameter count.

Group D: changing dropout from 0.1 to 0 gives 24.6; changing label smoothing from 0.1 to 0 gives 25.3. Group E: learned positional embeddings score 25.7, close to the sinusoidal base's 25.8. These results describe configuration tradeoffs in this setting: a larger FFN helps, the positional-encoding difference is small, and removing dropout causes a clear score decrease.

5.4 What the paper actually solves

The original Transformer primarily improves **information exchange and training parallelism in supervised sequence transduction**. When the full source sentence and paired target translations are available, it replaces recurrent or convolutional within-sequence computation with self-attention, shortens connections between distant positions, and obtains the reported WMT 2014 English-to-German and English-to-French results. It supplies an effective encoder-decoder architecture and training recipe. The main evidence is in Sections 3–6 and Tables 1–3.

Applying it in a system requires separating three questions: which information the architecture can connect, what the training data can teach it, and when the application requires an output under particular resource constraints. The paper primarily advances the first question and validates it on specific translation tasks. The following scenarios examine the additional conditions needed for limited data, real-time inputs, and long contexts.

5.5 Remaining problems and application scope

Research checked through 28 October 2026. The following 2025–2026 studies examine low-resource translation, simultaneous interpretation, and long contexts; FlashAttention illustrates an engineering limitation improved by later work. References [3]–[8] inform present-day applicability. Reference [1] supplies the original Transformer's experimental evidence.

5.5.1 Learning reliable translation from little parallel data

Scenario: a translation system for languages such as Akkadian or Manipuri may have only tens of thousands of sentence pairs. Its data, tokenizer coverage, and feasible model capacity differ substantially from the large English-to-German and English-to-French training settings.

Evidence: on approximately 21,000–50,000 sentence pairs, Signoroni et al. (2025) find a capacity/quality tradeoff: smaller Transformer configurations substantially reduce parameters while retaining much of the performance [3, Sections 3.1 and 4.1]. Song et al. (2026) observe low-resource quality gaps among tested small models on FLORES-200 and improve results with targeted distillation [4, Sections 2.3 and 4.3]. The original Transformer uses approximately 4.5 million English-to-German and 36 million English-to-French training pairs [1, Section 5.1]. Its experiment with about 40,000 sentences is English constituency parsing, with different tasks and supervision [1, Section 6.3].

Inference: the paper establishes effectiveness in its tested translation settings, but not a complete recipe for reliable translation from few sentence pairs. Low-resource transfer requires revisiting tokenization, capacity, regularization, and data augmentation or transfer, then evaluating the target language. This concerns learning and data conditions: the original paper does not validate this translation scenario; later configuration and distillation studies address parts of it.

5.5.2 Translating before the source sentence is complete

Scenario: live captions or real-time interpretation must produce output while input is still arriving. Waiting for every complete sentence may be unacceptable. Committing too early risks semantic or word-order errors, while waiting for more context adds latency.

Evidence: Zhang et al. (2026) add actions such as segmentation and redundancy removal to read/write decisions, studying quality/latency tradeoffs on ACL60/60 English-to-Chinese, English-to-German, and English-to-Japanese. Their framework remains text-to-text; joint speech-input and action decoding is future work [5, Sections 3–4 and 6]. Original Transformer encoder positions can read the complete source sentence; decoder cross-attention reads encoder outputs. Autoregressive beam search specifies no simultaneous policy for reading more input or committing a translation [1, Sections 3.2.3 and 6.1].

Inference: the paper translates an available source sequence; parallel training alone does not provide real-time interpretation. Streaming needs source-visibility constraints, read/write timing, and latency objectives; speech also needs acoustic processing. The original paper omits simultaneous decision-making. Later work improves incremental strategies, while real-time speech systems must evaluate input processing, decisions, and output delay together.

5.5.3 Fitting long-text computation into memory and time budgets

Scenario: moving from sentence translation to long documents or large code contexts increases the number of position pairs processed by dense attention. Training and context-processing time, together with GPU memory use, become practical constraints.

Evidence: Table 1 gives self-attention per-layer computation as $O(n^2 d)$, where n is sequence length and d representation width. Section 4 suggests restricting attention for very long sequences, but does not implement that proposal in this paper [1]. FlashAttention keeps attention exact while tiling computation and reducing GPU-memory transfers. Its Theorem 1 still gives $O(n^2 d)$ computation, while additional storage beyond inputs and outputs grows linearly with sequence length [6, Section 3.1, Theorem 1].

Inference: the original architecture shortens information-transfer paths but still pays for pairwise matching on long sequences. Later work reduces storage and memory-transfer overhead, while dense matching's computation remains a factor to assess. For a target length and hardware budget, the practical choice is whether to retain all connections or introduce local/sparse alternatives, then measure their effects on accuracy, memory, and time.

5.5.4 Reliably using relevant information in a long context

Scenario: retrieve scattered facts from a long document, understand code spanning several files, or answer using early information in a lengthy conversation. Accepting the text as input is only the first step; the system must also find and correctly use the relevant information.

Evidence: NoLiMa (2025) reduces literal overlap between questions and evidence. In Table 3, the tested GPT-4o scores 99.3 on the short-context base setting and 69.7 at 32K tokens [7, Sections 4.3–4.4, Table 3]. Bansal et al. (ICLR 2026) analyze attention-score dilution in long contexts and propose qTTT, which updates query projections at test time and improves performance on their evaluated long-context tasks [8, Sections 2.3 and 3, Figure 4]. The original paper's related evidence concerns short paths between positions and results on translation and constituency parsing; extrapolation to longer sequences is one motivation for sinusoidal encoding [1, Sections 3.5, 4, and 6].

Inference: the paper provides a mechanism for directly connecting distant information, but reliable selection and use of evidence also depend on learning and evaluation. Long-document applications need tests across lengths, evidence positions, and distractors, and may require long-context training or adaptation. The later model evaluations distinguish connectivity from effective use; methods such as qTTT improve some tested settings. The original paper establishes an architectural basis for connectivity. Effective use must still be confirmed for the particular task, training procedure, and evaluation results.

5.6 Choose reusable components for a specific application

Application	Reusable contribution	What must be added or checked
Supervised translation of complete sentences	Encoder-decoder, attention, and FFN	Target data, tokenization, and evaluation protocol
Target languages with little parallel data	An adjustable sequence-modeling architecture	Model capacity, data strategy, and target-language quality
Streaming translation	Representation and cross-language alignment modules	Source visibility, read/write policy, and latency
Long documents or conversations	Interaction between positions	Compute budget, retrieval, and effective use of evidence

Start from a concrete scenario, then design experiments around input conditions → gaps in existing methods → components this paper supplies → capabilities still requiring validation. With little parallel data, compare data strategies and capacity. For simultaneous translation, report both quality and latency. For long text, report resource use and accuracy in using evidence. This makes clear which Transformer contribution has been reused and which remaining problem a new method actually addresses.

6 References and source locations

[1] Vaswani, A., et al. Attention Is All You Need. NIPS 2017. https://arxiv.org/abs/1706.03762v7. Figures, tables, and page numbers in this report refer to this version. Architecture: Section 3, Figures 1–2; complexity: Section 4, Table 1; training: Section 5; main results and ablations: Section 6, Tables 2–3; attention visualization: appendix Figure 3.

[2] Gehring, J., Auli, M., Grangier, D., Yarats, D., Dauphin, Y. N. Convolutional Sequence to Sequence Learning. ICML 2017, PMLR 70:1243–1252. https://proceedings.mlr.press/v70/gehring17a.html. Reference [9] in the main paper; Sections 3.1–3.3 support the baseline mechanism comparison.

[3] Signoroni, E., Rychly, P., Signoroni, R. Efficient Architectures For Low-Resource Machine Translation. LowResNLP 2025, pp. 39–64. https://aclanthology.org/2025.lowresnlp-1.6/. Checked Sections 3.1 and 4.1 for low-resource data conditions and configuration tradeoffs.

[4] Song, Y., et al. Are Small Language Models the Silver Bullet to Low-Resource Languages Machine Translation? LoResMT 2026, pp. 1–26. https://aclanthology.org/2026.loresmt-1.1/. Checked Sections 2.3 and 4.3 for multilingual evaluation and improvements through distillation.

[5] Zhang, Q., Yang, Z., Nakamura, S. Redefining Machine Simultaneous Interpretation: From Incremental Translation to Human-Like Strategies. Findings of ACL 2026, pp. 12554–12577. https://aclanthology.org/2026.findings-acl.611/. Checked Sections 3–4 and 6 for interpretation actions and application scope.

[6] Dao, T., et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. 2022. arXiv:2205.14135v2. https://arxiv.org/abs/2205.14135v2. Checked Section 3.1, Theorem 1, and Section 3.2 to distinguish computational complexity from storage overhead.

[7] Modarressi, A., et al. NoLiMa: Long-Context Evaluation Beyond Literal Matching. ICML 2025; this report uses arXiv:2502.05167v3 (9 July 2025). https://arxiv.org/abs/2502.05167v3. Checked Sections 4.3–4.4 and Table 3; the reported numbers refer to that version and its tested model.

[8] Bansal, R., et al. Let's (not) just put things in Context: Test-time Training for Long-context LLMs. ICLR 2026. https://proceedings.iclr.cc/paper_files/paper/2026/hash/b619cd6dcc986856b8a8da2b08d89396-Abstract-Conference.html. Checked Sections 2.3 and 3 and Figure 4 for test-time query-projection updates and long-context utilization.